CCE unit = 4

Tree

Page No: 26
Date.

kth largest Element in a Binary search Tree (BST) modification.

problem statement.

Develop a C++ program To find The kth largest element in a Binary search Tree (BST). The program accept distinct integer values, inserts Them into BST. and asks The user to enter one or more k value if validates every k value and reports when k is outside The valid range. During The search. The program display The mode. visited in reverse inorder order (Right - root - left) stop when The kth largest element is found. and records how many Node were visited. The program also compare the Number of visited Node for different k values.

Introduction

A Binary search tree is non-linear data structure that stores value in a hieachical form. for each Node. values smaller than The Node are placed in its left subtree and values greater Than The node are placed in its right subtree. This arrangement supports efficient searching and ordered processing of data.

Page No:
Date.

objectives

- Construct a BST from distinct integer value.
- Find the kth largest using reverse inorder Traversal
- Validate k and display an error for value outside to n.
- Display the visited nodes in reverse inorder order for each valid query.
- Count and compare visited Node for the different k values.

Algorithm

1. Start
2. Read The Number of elements n. if n is not positive display and error and stop.
3. Read a distinct integer values and insert Them into The BST.
4. Display The meny and ask how mank k queries The user wants to test
5. for each query . read k.
6. if k>1 or k>n, display. "invalid value of k". and Continue To The next query.
7. for a valid k. set The visit Counter To zero and begin reverse inorder Traversal.
8. visit The Right subtree first. if Traversal has not finished. visit The current node display its value. and increase The visit Counter.
9. when The visit Counter reaches k. store The current Node as The kth largest element and stop that quary inversal.

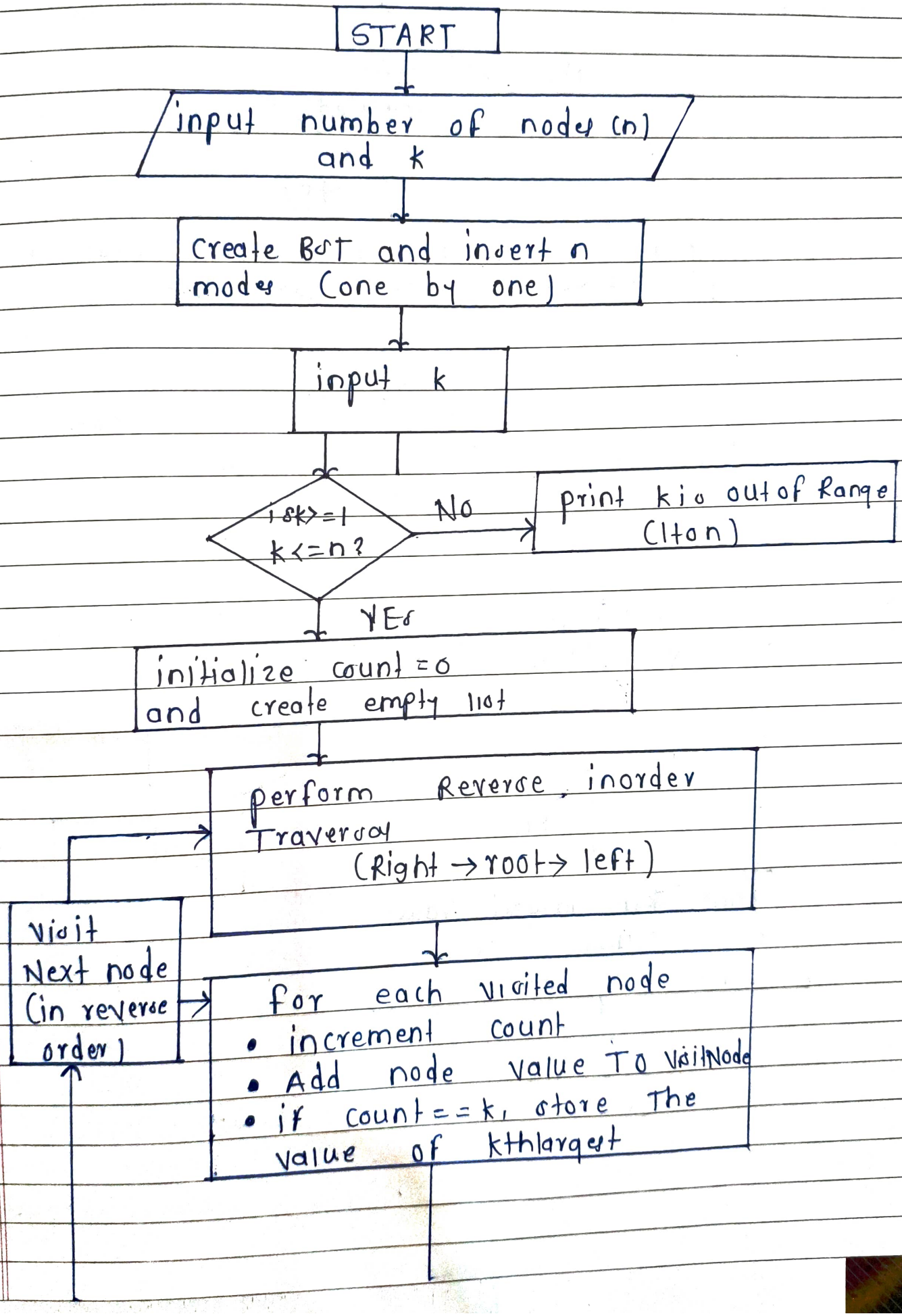
Page No:
Date.

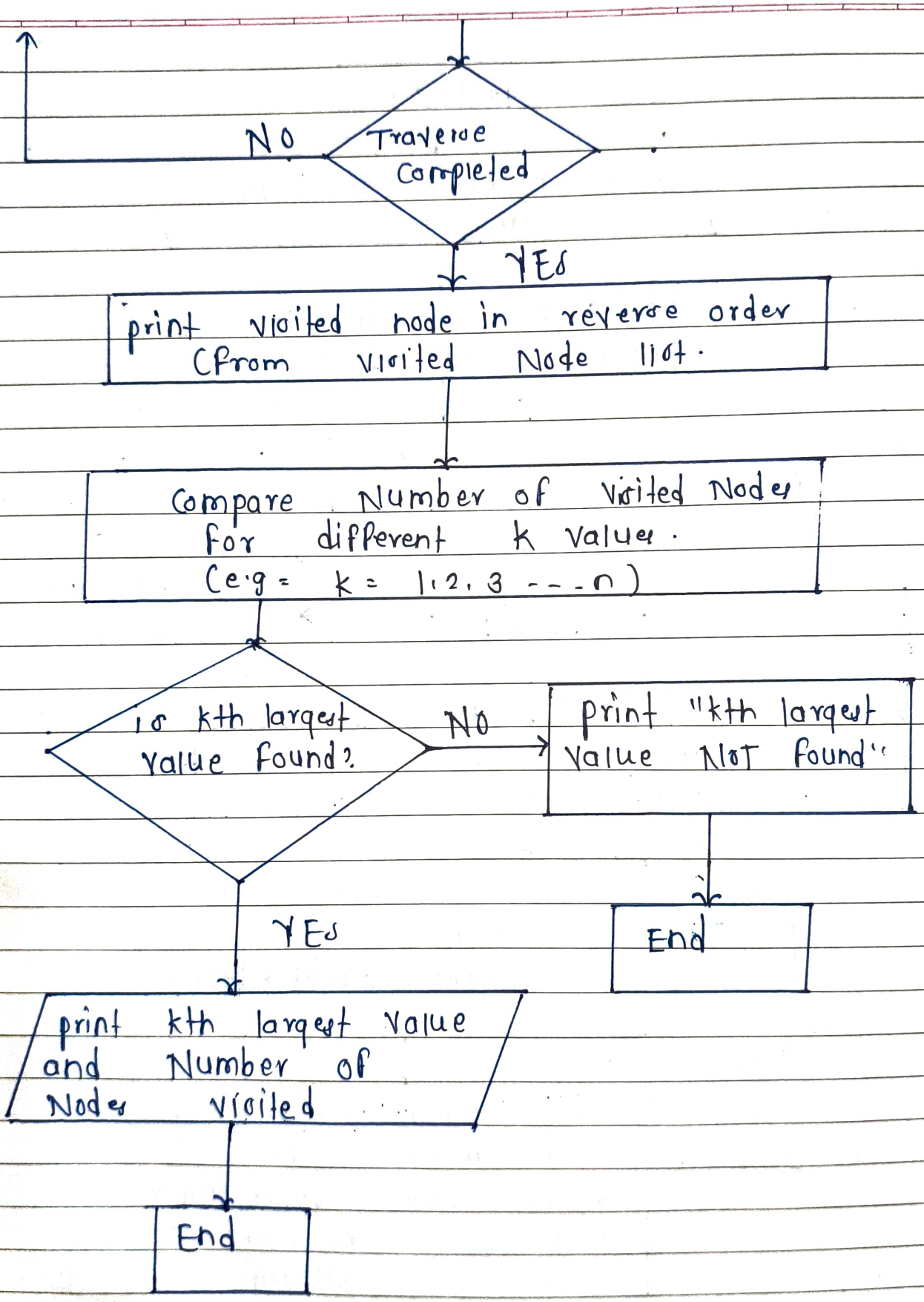
10. Visit The left subtree only if The answer has Not yet been found.

11. Display The kth largest element. The visited Node sequence. and The Number of visited Nodes.

12: After all queries: compare The visit counter for valid k value. Then stop.

flowchart


START
input number of nodes (n) and k
Create BST and insert n nodes (one by one)
input k
1<=k<=n?
No
print k is out of Range (1 to n)
YES
initialize count = 0 and create empty list
perform Reverse inorder Traversal (Right → root → left)
for each visited node
• increment count
• Add node value To VisitNode
• if count == k, store The value of kthlargest
Visit Next node (in reverse order)


NO
Traverse Completed
YES
print visited node in reverse order (from visited Node list.
Compare Number of visited Node for different k value. (e.g = k = 1, 2, 3 ---n)
is kth largest value found?
NO
print "kth largest value NOT found"
End
YES
print kth largest value and Number of Node visited
End

```
#include <iostream>
using namespace std;

struct Node
{
  int data;
  Node* left;
  Node* right;
};

Node* createNode(int value)
{
  Node* newNode = new Node();
  newNode->data = value;
  newNode->left = NULL;
  newNode->right = NULL;
  return newNode;
}

Node* insert(Node* root, int value)
{
  if (root == NULL)
    return createNode(value);

  if (value < root->data)
    root->left = insert(root->left, value);
  else if (value > root->data)
    root->right = insert(root->right, value);
```

```
    return root;
}

void displayInorder(Node* root)
{
    if (root == NULL)
        return;

    displayInorder(root->left);
    cout << root->data << " ";
    displayInorder(root->right);
}

void findKthLargest(Node* root, int& k, int& answer)
{
    if (root == NULL || k == 0)
        return;

    findKthLargest(root->right, k, answer);

    if (k > 0)
    {
        k--;

        if (k == 0)
        {
            answer = root->data;
```

```
        return;
      }
    }

    findKthLargest(root->left, k, answer);
}

int main()
{
    Node* root = NULL;
    int n, value, k, answer = -1;

    cout << "Kth Largest Element in BST\n";

    cout << "Enter number of nodes: ";
    cin >> n;

    if (n <= 0)
    {
      cout << "Invalid number of nodes.";
      return 0;
    }

    cout << "Enter BST elements:\n";

    for (int i = 0; i < n; i++)
    {
      cin >> value;
```

```
        root = insert(root, value);
    }

    cout << "\nBST elements in ascending order: ";
    displayInorder(root);

    cout << "\nEnter the value of K: ";
    cin >> k;

    if (k <= 0 || k > n)
    {
        cout << "Invalid K value.";
        return 0;
    }

    findKthLargest(root, k, answer);

    cout << "Kth Largest Element = " << answer << endl;

    return 0;
}
```

